

Créer des vagues d'ennemis avec des listes

Dans les jeux, il est courant de faire apparaître des vagues d'ennemis à des intervalles réguliers. Une façon efficace de gérer cela dans Unity est d'utiliser des listes pour stocker les types d'ennemis et leurs positions d'apparition. Nous allons utiliser les méthodes plus avancées pour manipuler les listes vues précédemment.

1.1 Méthodes couramment utilisées avec les listes

- `Add(item)` : Ajoute un élément à la fin de la liste.
- `Remove(item)` : Supprime la première occurrence de l'élément spécifié de la liste.
- `Clear()` : Supprime tous les éléments de la liste.
- `Count` : Propriété qui retourne le nombre d'éléments dans la liste.
- `Insert(index, item)` : Insère un élément à l'index spécifié.
- `RemoveAt(index)` : Supprime l'élément à l'index spécifié.
- `Contains(item)` : Retourne vrai si l'élément spécifié est présent dans la liste.

1.2 Trier une liste d'ennemis en fonction de leur niveau de difficulté

Vous pouvez trier une liste d'ennemis en fonction de leur niveau de difficulté en utilisant la méthode `Sort()` avec un comparateur personnalisé.

```
//Script d'un ennemi
using System;
using System.Collections.Generic;

public class Ennemi
{
    public string nom;
    public int niveauDifficulte;
    //... reste du code d'un ennemi
}

```csharp
using System;
using System.Collections.Generic;
public class EnnemiManager : MonoBehaviour
{
 public List<GameObject> listeEnnemis = new List<GameObject>();

 void TrierEnnemisParDifficulte()
 {
 // Trier les ennemis par niveau de difficulté
 listeEnnemis.Sort((e1, e2) {
 Ennemi ennemi1 = e1.GetComponent<Ennemi>();
 Ennemi ennemi2 = e2.GetComponent<Ennemi>();
```

```
 if (ennemi1.niveauDifficulte < ennemi2.niveauDifficulte) return -1;
 if (ennemi1.niveauDifficulte > ennemi2.niveauDifficulte) return 1;
 return 0;
 });
}
```

### 1.3 Choisir le dernier ennemi dans la liste juste

Lorsque possible, il est toujours plus optimal de retirer le dernier élément d'une liste pour éviter de réindexer tous les éléments suivants. Vous pouvez accéder au dernier ennemi dans une liste en utilisant l'index `Count - 1` et en utilisant `RemoveAt` pour le supprimer.

Vous continuer de retirer un élément de la liste jusqu'à ce qu'elle soit vide. Vous pouvez passer ensuite à la vague suivante ou au niveau suivant.

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class EnnemiManager : MonoBehaviour
{
 public List<GameObject> listeEnnemis = new List<GameObject>();
 public List<Transform> listePositionsSpawn = new List<Transform>();

 void Start()
 {
 InvokeRepeating("GenererEnnemi", 2f, 5f); // Génère un ennemi toutes les 5 secondes après un délai
 // initial de 2 secondes
 }

 void GenererEnnemi()
 {
 if (listeEnnemis.Count > 0)
 {
 //Choisir un point de spawn au hasard
 Transform pointCreation = listePositionsSpawn[Random.Range(0, listePositionsSpawn.Count)];

 // Accéder au dernier ennemi
 // Supprimer le dernier ennemi de la liste
 GameObject dernierEnnemi = listeEnnemis[listEnnemis.Count - 1];
 listeEnnemis.RemoveAt(listEnnemis.Count - 1);

 GameObject cloneEnnemi = Instantiate(dernierEnnemi, pointCreation.position,
 // pointCreation.rotation);
 cloneEnnemi.SetActive(true);
 }else{
```

```
 Debug.Log("Tous les ennemis ont été retirés de la liste !");
 // Choisir la scène de fin de jeu ou passer à la vague suivante
 SceneManager.LoadScene("FinJeu");
 }
}
}
```